

The use of copyrighted materials in all formats, including the creation, online delivery, and use of digital copies of copyrighted materials, must be in compliance with U.S. Copyright Law (<http://www.copyright.gov/title17/>). Materials may not be reproduced in any form without permission from the publisher, except as permitted under U.S. copyright law. **Copyrighted works are provided under Fair Use Guidelines only to serve personal study, scholarship, research, or teaching needs.**

Maurizio Gabbrielli and Simone Martini

Programming Languages: Principles and Paradigms

 Springer

Prof. Dr. Maurizio Gabbrielli
Università di Bologna
Bologna
Italy

Prof. Dr. Simone Martini
Università di Bologna
Bologna
Italy

Series editor
Ian Mackie

Advisory board

Samson Abramsky, University of Oxford, UK
Chris Hankin, Imperial College London, UK
Dexter Kozen, Cornell University, USA
Andrew Pitts, University of Cambridge, UK
Hanne Riis Nielson, Technical University of Denmark, Denmark
Steven Skiena, Stony Brook University, USA
Iain Stewart, University of Durham, UK
David Zhang, The Hong Kong Polytechnic University, Hong Kong

ISSN 1863-7310

ISBN 978-1-84882-913-8

e-ISBN 978-1-84882-914-5

DOI 10.1007/978-1-84882-914-5

Springer London Dordrecht Heidelberg New York

British Library Cataloguing in Publication Data

A catalogue record for this book is available from the British Library

Library of Congress Control Number: 2010922995

Translated from the original Italian edition: "Linguaggi di programmazione: principi e paradigmi", published in 2006 by McGraw-Hill Companies, Publishing Group Italia.

© Springer-Verlag London Limited 2010

Apart from any fair dealing for the purposes of research or private study, or criticism or review, as permitted under the Copyright, Designs and Patents Act 1988, this publication may only be reproduced, stored or transmitted, in any form or by any means, with the prior permission in writing of the publishers, or in the case of reprographic reproduction in accordance with the terms of licenses issued by the Copyright Licensing Agency. Enquiries concerning reproduction outside those terms should be sent to the publishers.

The use of registered names, trademarks, etc., in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant laws and regulations and therefore free for general use.

The publisher makes no representation, express or implied, with regard to the accuracy of the information contained in this book and cannot accept any legal responsibility or liability for any errors or omissions that may be made.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

```

hanoi(s(N), A, B, C, Moves):-
    hanoi(N, A, C, B, Moves1),
    hanoi(N, C, B, A, Moves2),
    append(Moves1, [move(A,B)|Moves2], Moves).

```

The call `hanoi(n, A, B, C, Move)`, where `n` is a (term which represents a) natural number, solves the problem of moving the tower of discs from `A` to `B` using `C` as an auxiliary pole. The solution is contained in the `Move` variable which, when the computation terminates, is instantiated with the list containing move which contribute to the solution. Every move is represented by a term of the form `move(X, Y)` to indicate the move of the top disc of pole `X` to pole `Y`.

This amazingly simple program amply shows the power of logic programming and also recursive reasoning. As usual, we give its logical interpretation. The first clause is clear: if we have just one disc, all we have to do is to move it from `A` from `B`. The reading of the second clause also is quite intuitive. If `Moves1` is the list of moves that solves the problem of moving a tower of `N` discs from `A` to `C` using `B` as an auxiliary pole and `Moves2` is the list of moves needed to move the tower of `N` disc from `C` to `B` using `A` as an auxiliary pole, to solve our problem in the case of `N+1` (or `s(N)`) discs, clearly we would have to do the following: first, execute all the moves in `Moves1`, then move from `A` to `B` (with the move `move(A, B)`) the `N+1`st disc in `A` which, being the largest, must be the last (at the bottom) in `B` as well. Finally, execute all the moves in `Moves2`. This is exactly what is done by the `append` predicate which therefore will use the variable `Moves` to provide the solutions to this problem.

12.5 Extensions

Until now, we have seen pure logic programming and some, highly partial, aspects of PROLOG. In this last section, we will briefly outline each of the numerous extensions to the pure formalism, each of which would require an entire chapter to do it justice. In the bibliographical notes we have included some references for those wishing to learn more.

12.5.1 Prolog

PROLOG is a language that is much richer than it might appear from what has been said so far. Let us recall that, as already seen, this language differs from logic programming in its adoption of precise rules for selecting the next atom to be rewritten (left to right) and for selecting the next clause to use (top to bottom, according to the program text).

In addition to these, there are other important differences with the theoretical formalism. We will list some of these without pretending to be exhaustive.

Arithmetic A real language cannot allow itself the luxury of using a completely symbolic arithmetic in which natural numbers are represented as successors of zero and there are no predefined operators.

In PROLOG, there exist therefore integers and reals (as floating point) as predefined data structures and various operators for manipulating them. These include:

- The usual arithmetic operators $+$, $-$, $*$ and $//$ (integer division).
- Arithmetic comparison operators such as $<$, $<=$, $>=$, $=$, $=$ (equal), $\neq$ (different).
- An evaluation operator called `is`.

Unlike other symbols (function and predicate) used in PROLOG, these symbols use infix notation for ease of use. There are however numerous delicate aspects that require a deep knowledge to avoid simple errors which the programmer used to imperative languages can easily cause.

First, comparison operators always require that their operands are ground arithmetic expressions (they cannot contain variables). Thus, while we have:²⁰

```
?- 3*2 == 1+5
Yes
? 4 > 5+2
no
```

if we use terms that are not arithmetic expressions or which contain variables, error will be signalled, as in:

```
?- 3 > a
error in arithmetic expression: a is not a number
?- X == 3+5
instantiation fault.
```

The last example is particularly disturbing. What we want to say is that the evaluation of the arithmetic expression $3+5$ produces the value 8 which will therefore be bound to the variable X . This cannot be done using `==`, as seen above, nor can it be done using equality between terms as considered in the previous paragraph. If we write $X = 3+5$, indeed, the effect is that of binding X to the term $3+5$ (rather than the value 8). To avoid this problem, the expression evaluation operator `is` is introduced. This operator allows us to obtain the effect we desire; `s is t` indeed *unifies* s with the *value* of the ground arithmetic expression t (if t is not a ground arithmetic expression, we have an error). The following are some examples of the use of `is`:

```
?- X is 3+5;
X = 8
?- 8 is 3 + 5
```

²⁰As said above, the line with `?-` returns the query that we want to evaluate and the following one is the PROLOG interpreter's reply.

```

Yes
?- 6 is 3 * 3
no
?- X is Y* 2
error in arithmetic expression: Y*2 is not a number

```

The use of `is`, necessary to be able to evaluate an expression, makes the use of arithmetic in PROLOG rather complicated. For example, given the following program:

```

evaluate(0,0).
evaluate(s(X), Val+1) :- evaluate(X, Val)

```

The goal `evaluate(s(s(s(0))), X)` does not compute the value 3 for `X`, as perhaps would have been expected but the term `0+1+1+1` (see Exercise 9 at the end of this chapter, too).

Cut PROLOG provides various constructs for interaction with the abstract PROLOG machine's interpreter so that the normal flow of control can be modified. Among these, one of the more important (and most discussed) is *cut*. This is an argument-free predicate, written as an exclamation mark, which allows the programmer to eliminate some of the possible alternatives produced during evaluation, with the aim of increasing execution efficiency. It is used when we are sure that, when a condition is satisfied, the other clauses in the program are no longer useful. For example, the following program computes the minimum of two values, if a comparison condition is true, the other is necessarily false, so we can use `!` to express the fact that once the first clause has been used, there is no need to consider the second.

```

minimum(X, Y, X) :- X < Y, !.
minimum(X, Y, Y) :- X > Y.

```

Alternatively, if we are interested only in testing whether a value appears at least once in a list, we can use the following modification to the `member` program:

```

member(X, [X | _]) :- !.
member(X, [_ | _]) :- member(X, _).

```

In general, the meaning of `cut` is the following. If we have n clauses to define the predicate p

```

p(S1) :- A1.
...
p(Sk) :- B, !, C.
...
p(Sn) :- An.

```

if, during the evaluation of the goal $p(t)$, we find the k th clause in the list being used, we have the following cases:

1. If an evaluation of B^{21} fails, then we proceed by trying the $k + 1$ st clause.
2. If, instead, the evaluation of B succeeded, then $!$ is evaluated. It succeeds (it always does) and the evaluation proceeds with C . In the case of backtracking, however, all the alternative ways of computing B are eliminated, as well as are eliminated all the alternatives provided by the clauses from the k th to the n th to compute $p(t)$.

There is no need to say that cut, in addition to not being easy to use, eliminates a good part of the declarativeness of a program.

Disjunction If we want to express the disjunction of two goals, $G1$ and $G2$, that is the fact that it is sufficient that at least one of the two succeeds, we can use two clauses of the form:

```
p(X) :- G1.
p(X) :- G2.
```

with the goal $p(X)$. The same effect can be obtained writing $G1;G2$, where we have used the predefined predicate “;”, which represents disjunction.

If-then-else The traditional construct from imperative languages:

```
if B then C1 else C2
```

is provided in PROLOG as a built-in with syntax $B \rightarrow C1;C2$. The if then else can be implemented using the cut as follows:

```
if_then_else(B, C1, C2) :- B, !, C1.
if_then_else(B, C1, C2) :- C2.
```

In effect, this is the internal definition of the $B \rightarrow C1;C2$ construct.

Negation Thus far, we have seen that in the body of a clause, only “positive” atomic formulae can be used; in other words, they cannot be negated.²² Often, however, it can also be useful to use negated atomic formulae. For example, let us consider the following program, *flights*:

```
direct_flight(bologna, paris).
direct_flight(bologna, amsterdam).
direct_flight(paris, bombay).
direct_flight(amsterdam, moscow).
flight(X, Y) :- direct_flight(X, Y).
flight(X, Y) :- direct_flight(X, Z), flight(Z, Y).
```

²¹Here, B , just like A_i and C , is considered as an arbitrary goal and not necessarily an atomic one.

²²Here, we refer, clearly, to the $H : -A_1, \dots, A_n$ notation. If, instead, we consider a clause as a disjunction of literals, the atoms in the body are negated, given that the preceding representation is equivalent to $\neg A_1 \vee \dots \vee \neg A_n \vee H$.

Here, we have a series of facts defining the `direct_flight` predicate, which denotes the existence of a connection without stopover between two destinations. Then, we have the `flight` predicate which defines a flight, possibly with intermediate stops, between two locations. This will be a direct flight (clause 5) or rather a direct flight for a stopover different from the one that is the destination, followed by a flight from this stopover to the destination (clause 6). With this program, we can check the existence of a flight, or we can ask which destinations can be reached from a given airport:

```
?- flight(bologna, bombay)
Yes
?- direct_flight(bologna, moscow)
no
?- flight(bologna, X)
X = paris
```

However, we fail to express the fact that there exists only non-direct flights because we do not know how to express the fact that there *does not* exist a direct flight. To do this, we require negation. If we write:

```
indirect_flight(X, Y) :- flight(X, Y), not direct_flight(X, Y).
```

we mean to say that there exists an indirect flight between X and Y if there exists a flight (between X and Y) and there exist no direct flight. With this definition, we have:

```
?- indirect_flight(bologna, bombay)
Yes
?- indirect_flight(bologna, paris)
no
```

These results can be explained as follows. The PROLOG interpreter evaluates a goal such as `not G` by trying to evaluate the un-negated goal `G`. If the evaluation of this goal terminates (possibly after backtracking) with failure, then the goal `not G` succeeds. If, on the other hand, the goal `G` has a computation that terminates with success, then that of `not G` fails. Finally, if the evaluation of `G` does not terminate, then `not G` fails to terminate. This type of negation is called “negation as failure”,²³ in that, exactly, it interprets the negation of a goal in terms of failure of the un-negated goal. Note that, because of infinite computations,²⁴ this type of negation differs from the negation in classical logic, given that the lack of success of `G` is not equivalent to the success of the negated version `not G`. For example, neither the goal `p` nor the goal `not p` succeed in the program `p :- p`.

²³To be more precise, negation as failure, as defined in the theoretical model of logic programming, has a behaviour that is slightly different from that described because of the incompleteness of the PROLOG interpreter.

²⁴And also non-ground goals, which we will not consider here.